

GUIDE PROFESSIONNEL 2026

Angular 22 + Spring Boot 4.1 + Java 25 LTS + Flutter 3.47
Microservices, DDD, sécurité, données, événements, tests et DevOps

Format	Manuel d'architecture et cahier d'implémentation
Niveau	Avancé - objectif architecte logiciel / solution
Projet fil rouge	Plateforme multi-tenant de commerce interentreprises
Référence versions	23 août 2026 - versions stables vérifiées
Approche	Microservices directs, vertical slices, production-first

Ce document est autonome. Il ne dépend d'aucun projet existant et décrit ce qu'il faut créer dans un nouveau dépôt.

Comment utiliser ce guide

Ce manuel n'est pas un tutoriel de syntaxe. Il entraîne la capacité à transformer un problème métier en système distribué exploitable. Chaque fiche correspond à une décision ou à un incrément que vous devez réellement produire. Implémentez dans l'ordre, conservez les preuves et soutenez vos choix comme lors d'une architecture review.

Règle de travail

Pour chaque fiche: 1) expliciter l'hypothèse, 2) écrire un test ou une preuve attendue, 3) implémenter le minimum, 4) observer, 5) documenter la décision. Ne passez pas à la phase suivante si les critères de sortie ne sont pas vérifiés.

Objectif final

- Un portail Angular pour administrateurs, vendeurs et acheteurs B2B
- Une application Flutter pour catalogue, approbation et suivi de commande
- Des microservices Spring Boot indépendants autour des contextes métier
- Une plateforme Kubernetes livrée par GitOps et provisionnée par Terraform
- Des SLO, runbooks, tests de sécurité, plan de reprise et coûts unitaires mesurés

Définition de Done globale

- Le code compile et les tests pertinents passent
- Les contrats API/événements restent compatibles
- Les données sont isolées par tenant jusque dans cache, jobs et exports
- Le composant expose santé, métriques, logs structurés et traces
- Le déploiement, le rollback et la restauration sont documentés et testés
- La décision comporte coût, risques, alternatives et date de revue

Stack de référence vérifiée

Versions stables disponibles au 23 août 2026. Le code que vous créerez plus tard doit pinner ses dépendances et vérifier les matrices de compatibilité officielles; 'latest' n'est jamais une stratégie de production.

Composant	Baseline	Usage architectural
Angular	22 (CLI local 22.1.3)	Web standalone, Signals, Signal Forms, zoneless
Spring Boot	4.1.1	Services Java, Actuator, Security, Data
Java	25 LTS	Runtime cible; éviter les preview features au coeur
Flutter	3.47 stable	Client mobile iOS/Android
PostgreSQL	18.6	Base par service, RLS pour isolation
Apache Kafka	4.3.1	Événements d'intégration et streaming
Keycloak	26.7.2	OIDC/OAuth2, fédération entreprise
Kubernetes	1.36.2	Orchestration et politiques runtime
Observabilité	OpenTelemetry	Traces, métriques et logs corrélés

Choix à ne pas copier aveuglément

PostgreSQL 18 et Kafka 4.3 sont les versions stables les plus récentes, mais votre cloud managé peut supporter une version antérieure. La bonne décision est la version stable supportée par votre opérateur, avec correctifs de sécurité, chemin d'upgrade et test de charge. Même principe pour Spring Cloud, plugins Flutter et bibliothèques Angular.

Architecture cible

La cible sépare clients, edge, domaines, données et plateforme. Chaque service possède sa base logique, son pipeline, son SLO et son ownership. Kafka propage les faits; il ne devient pas une base métier universelle.

CLIENTS	Angular 22 Admin/Buyer Flutter 3.47 Mobile
EDGE	CDN/WAF -> API Gateway/BFF -> OIDC, quotas, trace
DOMAINES	Tenant Catalog Pricing Inventory Order Payment Billing Notification
ÉVÉNEMENTS	Kafka + Schema Registry + Outbox/CDC + consommateurs idempotents
DONNÉES	PostgreSQL par service Redis ciblé Object storage Search optionnelle
PLATEFORME	Kubernetes GitOps Terraform Secrets OpenTelemetry Policy as Code

Flux de commande principal

- Angular/Flutter obtient un token OIDC avec PKCE.
- Gateway valide contraintes edge et transmet une identité signée; le service revalide JWT et audience.
- Order crée la commande et son événement Outbox dans une transaction PostgreSQL.
- Inventory réserve; Approval décide; Payment traite; la saga compense en cas d'échec.
- Billing produit les documents; Notification informe; les read models alimentent les interfaces.
- OpenTelemetry relie HTTP, base, Kafka et appels PSP dans une trace unique.

Context map et ownership

Contexte	Responsabilité	Données possédées	Événements clés
Tenant	Cycle de vie organisation	tenant, plan, entitlements	TenantActivated
Catalog	Information produit	product, variant, category	ProductPublished
Pricing	Prix contractuels	price list, rule	PriceListActivated
Inventory	Disponibilité	stock, reservation	StockReserved
Order	Engagement commercial	cart, order, approval	OrderApproved
Payment	Transaction PSP	payment intent, ledger ref	PaymentCaptured
Billing	Documents financiers	invoice, credit note	InvoiceIssued
Notification	Livraison messages	template, delivery	NotificationSent

Règle d'ownership

Un service n'accède jamais directement aux tables d'un autre. Pour une lecture immédiate, il utilise un contrat synchrone avec timeout. Pour intégrer un changement durable, il consomme un événement versionné et construit son propre read model. Une donnée dupliquée est acceptable si son propriétaire et sa fraîcheur sont explicites.

Sommaire opérationnel

Chaque partie commence par une architecture review puis six fiches d'implémentation. Les pages suivantes sont conçues comme un backlog exécutable.

- 01. Posture d'architecte et cadrage du produit** - Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable.
- 02. DDD stratégique et découpage en bounded contexts** - Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices.
- 03. Architecture microservices et plateforme** - Définir des services autonomes, opérables et découplés sans créer un monolithe distribué.
- 04. Spring Boot 4.1 et architecture hexagonale** - Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables.
- 05. Multi-tenancy et isolation des données** - Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants.
- 06. Contrats API, intégration et cohérence** - Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée.
- 07. Kafka, Outbox, Saga et traitement fiable** - Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités.
- 08. Sécurité OAuth2/OIDC et Zero Trust** - Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius.
- 09. Catalogue, pricing, stock et commande B2B** - Implémenter le coeur métier avec invariants explicites et frontières cohérentes.
- 10. Paiement, facturation et abonnement SaaS** - Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit.
- 11. Angular 22: architecture web professionnelle** - Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals.
- 12. Flutter 3.47: application mobile cohérente** - Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular.
- 13. Tests et stratégie de qualité** - Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques.
- 14. Résilience et performance distribuée** - Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance.
- 15. Observabilité, SRE et opérations** - Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition.
- 16. Docker, Kubernetes et sécurité supply chain** - Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes.
- 17. CI/CD, GitOps et Infrastructure as Code** - Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé.
- 18. Gouvernance, coûts et évolution d'architecture** - Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût.
- 19. Capstone: trajectoire d'implémentation complète** - Assembler les décisions en une exécution progressive, démontrable et exploitable en production.

PARTIE 01

Posture d'architecte et cadrage du produit

Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable.

Principes directeurs

- Commencer par les risques et les qualités attendues, pas par les frameworks
- Relier chaque décision à une hypothèse métier
- Conserver une architecture évolutive, observable et économiquement soutenable

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Sur-architecture avant product-market fit	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Objectifs non mesurables	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Choix techniques sans propriétaire	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Vision B2B et proposition de valeur

But. Définir vendeurs, acheteurs, administrateurs, approbateurs et opérateurs; distinguer abonnement SaaS et flux commercial.

Décision à prendre

Définissez la frontière et le niveau de service de **Vision B2B et proposition de valeur**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour vision b2b et proposition de valeur.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir vendeurs, acheteurs, administrateurs, approbateurs et opérateurs; distinguer abonnement SaaS et flux commercial.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Vision produit, personas et carte de valeur est versionné, relu et reproductible.
- Le parcours vision b2b et proposition de valeur possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Vision produit, personas et carte de valeur. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment vision b2b et proposition de valeur se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Exigences fonctionnelles

But. Cadrer catalogue contractuel, prix négociés, commandes, approbations, factures, paiements et notifications.

Décision à prendre

Définissez la frontière et le niveau de service de **Exigences fonctionnelles**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour exigences fonctionnelles.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Cadrer catalogue contractuel, prix négociés, commandes, approbations, factures, paiements et notifications.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Backlog de capacités métier priorisé est versionné, relu et reproductible.
- Le parcours exigences fonctionnelles possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Backlog de capacités métier priorisé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment exigences fonctionnelles se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Attributs de qualité

But. Exprimer disponibilité, latence, sécurité, auditabilité, évolutivité, portabilité et coût sous forme de scénarios.

Décision à prendre

Définissez la frontière et le niveau de service de **Attributs de qualité**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour attributs de qualité.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Exprimer disponibilité, latence, sécurité, auditabilité, évolutivité, portabilité et coût sous forme de scénarios.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Quality Attribute Scenarios est versionné, relu et reproductible.
- Le parcours attributs de qualité possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Quality Attribute Scenarios. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment attributs de qualité se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Hypothèses de charge

But. Modéliser 10 000 tenants, pics de consultation, commandes simultanées, taille des catalogues et saisonnalité.

Décision à prendre

Définissez la frontière et le niveau de service de **Hypothèses de charge**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour hypothèses de charge.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Modéliser 10 000 tenants, pics de consultation, commandes simultanées, taille des catalogues et saisonnalité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Modèle de capacité initial est versionné, relu et reproductible.
- Le parcours hypothèses de charge possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Modèle de capacité initial. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment hypothèses de charge se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Architecture fitness functions

But. Automatiser les règles de dépendance, contrats, sécurité, performance et conformité.

Décision à prendre

Définissez la frontière et le niveau de service de **Architecture fitness functions**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour architecture fitness functions.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Automatiser les règles de dépendance, contrats, sécurité, performance et conformité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Catalogue de fitness functions est versionné, relu et reproductible.
- Le parcours architecture fitness functions possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Catalogue de fitness functions. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment architecture fitness functions se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Roadmap par incréments

But. Découper foundation, MVP, scale et entreprise sans construire toutes les capacités au premier sprint.

Décision à prendre

Définissez la frontière et le niveau de service de **Roadmap par incréments**. La décision doit satisfaire l'objectif de la partie: Transformer une intention produit en contraintes mesurables, décisions réversibles et trajectoire livrable. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour roadmap par incréments.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Découper foundation, MVP, scale et entreprise sans construire toutes les capacités au premier sprint.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Roadmap à quatre horizons est versionné, relu et reproductible.
- Le parcours roadmap par incréments possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Roadmap à quatre horizons. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment roadmap par incréments se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 02

DDD stratégique et découpage en bounded contexts

Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices.

Principes directeurs

- Un bounded context possède son langage et son modèle
- Les événements expriment des faits métier au passé
- Une frontière de domaine n'est pas automatiquement un déploiement

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Microservices découpés par tables	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Modèle partagé global	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Dépendances cycliques entre équipes	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Domain storytelling

But. Raconter acquisition d'un tenant, création d'un catalogue, négociation de prix et cycle de commande.

Décision à prendre

Définissez la frontière et le niveau de service de **Domain storytelling**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour domain storytelling.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Raconter acquisition d'un tenant, création d'un catalogue, négociation de prix et cycle de commande.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Récits métier validés par les parties prenantes est versionné, relu et reproductible.
- Le parcours domain storytelling possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Récits métier validés par les parties prenantes. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment domain storytelling se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Event Storming

But. Faire émerger commandes, événements, politiques, agrégats, systèmes externes et points chauds.

Décision à prendre

Définissez la frontière et le niveau de service de **Event Storming**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour event storming.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Faire émerger commandes, événements, politiques, agrégats, systèmes externes et points chauds.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Mur d'événements versionné est versionné, relu et reproductible.
- Le parcours event storming possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Mur d'événements versionné. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment event storming se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Ubiquitous Language

But. Définir Product, Offer, PriceList, Buyer, Approval, Order, Invoice et Subscription sans ambiguïté.

Décision à prendre

Définissez la frontière et le niveau de service de **Ubiquitous Language**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour ubiquitous language.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir Product, Offer, PriceList, Buyer, Approval, Order, Invoice et Subscription sans ambiguïté.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Glossaire par contexte est versionné, relu et reproductible.
- Le parcours ubiquitous language possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Glossaire par contexte. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment ubiquitous language se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Bounded contexts

But. Séparer Tenant, Identity, Catalog, Pricing, Inventory, Order, Payment, Billing et Notification.

Décision à prendre

Définissez la frontière et le niveau de service de **Bounded contexts**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour bounded contexts.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Séparer Tenant, Identity, Catalog, Pricing, Inventory, Order, Payment, Billing et Notification.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Carte des contextes est versionné, relu et reproductible.
- Le parcours bounded contexts possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Carte des contextes. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment bounded contexts se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Context mapping

But. Qualifier upstream/downstream, customer-supplier, conformist, anticorruption layer et open-host service.

Décision à prendre

Définissez la frontière et le niveau de service de **Context mapping**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour context mapping.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Qualifier upstream/downstream, customer-supplier, conformist, anticorruption layer et open-host service.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Context map et contrats d'équipe est versionné, relu et reproductible.
- Le parcours context mapping possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Context map et contrats d'équipe. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment context mapping se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Agrégats et invariants

But. Choisir des frontières transactionnelles courtes autour de Product, PriceList, Order et Payment.

Décision à prendre

Définissez la frontière et le niveau de service de **Agrégats et invariants**. La décision doit satisfaire l'objectif de la partie: Aligner frontières métier, responsabilités d'équipe et unités de changement avant de dessiner les microservices. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour agrégats et invariants.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Choisir des frontières transactionnelles courtes autour de Product, PriceList, Order et Payment.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Modèle tactique initial est versionné, relu et reproductible.
- Le parcours agrégats et invariants possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Modèle tactique initial. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment agrégats et invariants se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Architecture microservices et plateforme

Définir des services autonomes, opérables et découplés sans créer un monolithe distribué.

Principes directeurs

- Database per service et ownership explicite
- API synchrones pour interaction immédiate, événements pour propagation
- Aucun appel distribué à l'intérieur d'une transaction locale

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Chaînes synchrones longues	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Bibliothèque métier partagée	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Déploiements couplés	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Topologie des services

But. Définir gateway, tenant, catalog, pricing, inventory, order, payment, billing et notification.

Décision à prendre

Définissez la frontière et le niveau de service de **Topologie des services**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour topologie des services.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir gateway, tenant, catalog, pricing, inventory, order, payment, billing et notification.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- C4 Container diagram est versionné, relu et reproductible.
- Le parcours topologie des services possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

C4 Container diagram. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment topologie des services se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

API Gateway et BFF

But. Centraliser edge concerns sans déplacer la logique métier; évaluer BFF web et mobile.

Décision à prendre

Définissez la frontière et le niveau de service de **API Gateway et BFF**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour api gateway et bff.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Centraliser edge concerns sans déplacer la logique métier; évaluer BFF web et mobile.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Contrat de l'edge est versionné, relu et reproductible.
- Le parcours api gateway et bff possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Contrat de l'edge. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment api gateway et bff se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Service discovery

But. Comparer DNS Kubernetes, discovery applicatif et service mesh; choisir le minimum nécessaire.

Décision à prendre

Définissez la frontière et le niveau de service de **Service discovery**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour service discovery.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Comparer DNS Kubernetes, discovery applicatif et service mesh; choisir le minimum nécessaire.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- ADR de découverte est versionné, relu et reproductible.
- Le parcours service discovery possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

ADR de découverte. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment service discovery se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Configuration externalisée

But. Séparer configuration non secrète, secrets, feature flags et paramètres par tenant.

Décision à prendre

Définissez la frontière et le niveau de service de **Configuration externalisée**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour configuration externalisée.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Séparer configuration non secrète, secrets, feature flags et paramètres par tenant.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Modèle de configuration est versionné, relu et reproductible.
- Le parcours configuration externalisée possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Modèle de configuration. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment configuration externalisée se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Compatibilité des versions

But. Utiliser BOM, matrices officielles Spring Cloud et politiques d'upgrade régulières.

Décision à prendre

Définissez la frontière et le niveau de service de **Compatibilité des versions**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour compatibilité des versions.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Utiliser BOM, matrices officielles Spring Cloud et politiques d'upgrade régulières.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Politique de dépendances est versionné, relu et reproductible.
- Le parcours compatibilité des versions possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Politique de dépendances. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment compatibilité des versions se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Architecture repository

But. Choisir mono-repo contrôlé ou poly-repo; standardiser templates sans coupler les domaines.

Décision à prendre

Définissez la frontière et le niveau de service de **Architecture repository**. La décision doit satisfaire l'objectif de la partie: Définir des services autonomes, opérables et découplés sans créer un monolithe distribué. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour architecture repository.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Choisir mono-repo contrôlé ou poly-repo; standardiser templates sans coupler les domaines.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Structure des dépôts est versionné, relu et reproductible.
- Le parcours architecture repository possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Structure des dépôts. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment architecture repository se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 04

Spring Boot 4.1 et architecture hexagonale

Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables.

Principes directeurs

- Domaine sans annotations d'infrastructure
- Cas d'usage comme frontière transactionnelle
- Dépendances dirigées vers le cœur

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Entités jpa utilisées comme api	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Services fourre-tout	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Exceptions techniques exposées	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Squelette Maven Java 25

But. Configurer toolchain, Spring Boot BOM, plugins de qualité, tests et reproductibilité.

Décision à prendre

Définissez la frontière et le niveau de service de **Squelette Maven Java 25**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour squelette maven java 25.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Configurer toolchain, Spring Boot BOM, plugins de qualité, tests et reproductibilité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Parent de service standardisé est versionné, relu et reproductible.
- Le parcours squelette maven java 25 possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Parent de service standardisé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment squelette maven java 25 se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Ports et adaptateurs

But. Organiser domain, application, adapters.in et adapters.out avec règles de dépendance.

Décision à prendre

Définissez la frontière et le niveau de service de **Ports et adaptateurs**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour ports et adaptateurs.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Organiser domain, application, adapters.in et adapters.out avec règles de dépendance.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
public interface CreateOrderUseCase {
    OrderId handle(CreateOrder command);
}

public interface OrderRepository {
    Optional<Order> findById(TenantId tenantId, OrderId orderId);
    void save(Order order);
}

@Transactional
final class CreateOrderService implements CreateOrderUseCase {
    private final OrderRepository orders;
    private final EventOutbox outbox;
    public OrderId handle(CreateOrder cmd) {
        Order order = Order.create(cmd.tenantId(), cmd.buyerId(), cmd.lines());
        orders.save(order);
        outbox.appendAll(order.pullEvents());
        return order.id();
    }
}
```

Critères de sortie

- Architecture de packages est versionné, relu et reproductible.
- Le parcours ports et adaptateurs possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Architecture de packages. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment ports et adaptateurs se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Modèle de domaine riche

But. Encapsuler invariants, value objects Money, identifiants typés et transitions d'état.

Décision à prendre

Définissez la frontière et le niveau de service de **Modèle de domaine riche**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour modèle de domaine riche.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Encapsuler invariants, value objects Money, identifiants typés et transitions d'état.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
public record Money(BigDecimal amount, Currency currency) {
    public Money {
        Objects.requireNonNull(amount);
        Objects.requireNonNull(currency);
        if (amount.scale() > currency.getDefaultFractionDigits())
            throw new IllegalArgumentException("invalid monetary scale");
    }
    public Money add(Money other) {
        if (!currency.equals(other.currency)) throw new CurrencyMismatch();
        return new Money(amount.add(other.amount), currency);
    }
}
```

Critères de sortie

- Agrégats testés sans Spring est versionné, relu et reproductible.
- Le parcours modèle de domaine riche possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Agrégats testés sans Spring. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment modèle de domaine riche se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Cas d'usage

But. Séparer commandes et requêtes, transactions, autorisation métier et publication d'événements.

Décision à prendre

Définissez la frontière et le niveau de service de **Cas d'usage**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour cas d'usage.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Séparer commandes et requêtes, transactions, autorisation métier et publication d'événements.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Ports d'entrée explicites est versionné, relu et reproductible.
- Le parcours cas d'usage possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Ports d'entrée explicites. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment cas d'usage se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

REST avec Problem Details

But. Concevoir ressources, validation, idempotency, pagination par curseur et erreurs RFC 9457.

Décision à prendre

Définissez la frontière et le niveau de service de **REST avec Problem Details**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour rest avec problem details.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Concevoir ressources, validation, idempotency, pagination par curseur et erreurs RFC 9457.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
@PostMapping("/orders")
ResponseEntity<OrderView> create(
    @RequestHeader("Idempotency-Key") UUID key,
    @Valid @RequestBody CreateOrderRequest body,
    JwtAuthenticationToken principal) {
    var command = mapper.toCommand(body, TenantId.from(principal.getToken()));
    var result = idempotency.execute(command.tenantId(), key, body, () -> useCase.handle(command));
    return ResponseEntity.created(URI.create("/orders/" + result.id())).body(result);
}
```

Critères de sortie

- API REST cohérente est versionné, relu et reproductible.
- Le parcours rest avec problem details possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

API REST cohérente. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment rest avec problem details se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Persistence JPA

But. Mapper domaine et persistence model, maîtriser fetch plans, locking et migrations Flyway.

Décision à prendre

Définissez la frontière et le niveau de service de **Persistence JPA**. La décision doit satisfaire l'objectif de la partie: Implémenter chaque service Java avec un domaine indépendant de Spring et des adaptateurs remplaçables. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour persistence jpa.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Mapper domaine et persistence model, maîtriser fetch plans, locking et migrations Flyway.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Adaptateur PostgreSQL est versionné, relu et reproductible.
- Le parcours persistence jpa possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Adaptateur PostgreSQL. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment persistence jpa se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 05

Multi-tenancy et isolation des données

Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants.

Principes directeurs

- Le tenant provient d'une identité vérifiée
- Defense in depth: application plus base
- Tout index métier commence par tenant_id quand pertinent

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Header tenant falsifiable	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Fuite via cache	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Contexte threadlocal non nettoyé	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Choix du modèle

But. Comparer database-per-tenant, schema-per-tenant, shared schema et modèle hybride entreprise.

Décision à prendre

Définissez la frontière et le niveau de service de **Choix du modèle**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour choix du modèle.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Comparer database-per-tenant, schema-per-tenant, shared schema et modèle hybride entreprise.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- ADR multi-tenancy est versionné, relu et reproductible.
- Le parcours choix du modèle possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

ADR multi-tenancy. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment choix du modèle se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tenant resolution

But. Résoudre tenant depuis issuer et claims signés; contrôler cohérence URL, token et ressource.

Décision à prendre

Définissez la frontière et le niveau de service de **Tenant resolution**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tenant resolution.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Résoudre tenant depuis issuer et claims signés; contrôler cohérence URL, token et ressource.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- TenantContext sécurisé est versionné, relu et reproductible.
- Le parcours tenant resolution possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

TenantContext sécurisé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tenant resolution se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PostgreSQL RLS

But. Activer FORCE ROW LEVEL SECURITY, politiques USING/WITH CHECK et rôle applicatif non bypass.

Décision à prendre

Définissez la frontière et le niveau de service de **PostgreSQL RLS**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour postgresql rls.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Activer FORCE ROW LEVEL SECURITY, politiques USING/WITH CHECK et rôle applicatif non bypass.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
ALTER TABLE product ENABLE ROW LEVEL SECURITY;
ALTER TABLE product FORCE ROW LEVEL SECURITY;
CREATE POLICY product_tenant_policy ON product
  USING (tenant_id = current_setting('app.tenant_id', true)::uuid)
  WITH CHECK (tenant_id = current_setting('app.tenant_id', true)::uuid);
CREATE INDEX ix_product_tenant_status
  ON product (tenant_id, status, updated_at DESC);
```

Critères de sortie

- Migrations RLS est versionné, relu et reproductible.
- Le parcours postgresql rls possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Migrations RLS. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment postgresql rls se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Pools et transactions

But. Positionner SET LOCAL dans chaque transaction et éviter la contamination des connexions.

Décision à prendre

Définissez la frontière et le niveau de service de **Pools et transactions**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour pools et transactions.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Positionner SET LOCAL dans chaque transaction et éviter la contamination des connexions.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Intégration datasource est versionné, relu et reproductible.
- Le parcours pools et transactions possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Intégration datasource. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment pools et transactions se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Caches multi-tenant

But. Préfixer les clés, séparer quotas, éviter les objets globaux et tester les évictions.

Décision à prendre

Définissez la frontière et le niveau de service de **Caches multi-tenant**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour caches multi-tenant.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Préfixer les clés, séparer quotas, éviter les objets globaux et tester les évictions.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Convention de cache est versionné, relu et reproductible.
- Le parcours caches multi-tenant possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Convention de cache. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment caches multi-tenant se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests anti-fuite

But. Créer tests négatifs tenant A vers B pour REST, repository, Kafka, export et recherche.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests anti-fuite**. La décision doit satisfaire l'objectif de la partie: Garantir qu'aucune requête, tâche asynchrone ou métrique ne mélange les données de tenants. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests anti-fuite.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Créer tests négatifs tenant A vers B pour REST, repository, Kafka, export et recherche.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Suite d'isolation automatisée est versionné, relu et reproductible.
- Le parcours tests anti-fuite possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Suite d'isolation automatisée. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests anti-fuite se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 06

Contrats API, intégration et cohérence

Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée.

Principes directeurs

- Contract first pour les frontières publiques
- Idempotency sur toute commande rejouable
- Eventual consistency visible dans l'UX

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Breaking change silencieux	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Retry d'une opération non idempotente	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Transaction distribuée implicite	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

OpenAPI 3.1

But. Définir schémas, scopes, erreurs, pagination, exemples et règles de compatibilité.

Décision à prendre

Définissez la frontière et le niveau de service de **OpenAPI 3.1**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour openapi 3.1.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir schémas, scopes, erreurs, pagination, exemples et règles de compatibilité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Spécifications versionnées est versionné, relu et reproductible.
- Le parcours openapi 3.1 possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Spécifications versionnées. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment openapi 3.1 se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Versioning

But. Privilégier évolution additive, période de dépréciation et télémétrie d'usage avant suppression.

Décision à prendre

Définissez la frontière et le niveau de service de **Versioning**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour versioning.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Privilégier évolution additive, période de dépréciation et télémétrie d'usage avant suppression.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Politique de versioning est versionné, relu et reproductible.
- Le parcours versioning possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Politique de versioning. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment versioning se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Idempotency keys

But. Persister empreinte de requête, résultat et expiration avec contrainte d'unicité par tenant.

Décision à prendre

Définissez la frontière et le niveau de service de **Idempotency keys**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour idempotency keys.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Persister empreinte de requête, résultat et expiration avec contrainte d'unicité par tenant.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Middleware d'idempotence est versionné, relu et reproductible.
- Le parcours idempotency keys possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Middleware d'idempotence. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment idempotency keys se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Communication synchrone

But. Budgets de timeout, propagation trace, retries sélectifs et clients générés.

Décision à prendre

Définissez la frontière et le niveau de service de **Communication synchrone**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour communication synchrone.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Budgets de timeout, propagation trace, retries sélectifs et clients générés.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Client inter-service durci est versionné, relu et reproductible.
- Le parcours communication synchrone possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Client inter-service durci. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment communication synchrone se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

CQRS pragmatique

But. Séparer modèles de lecture uniquement quand charge ou modèle le justifie.

Décision à prendre

Définissez la frontière et le niveau de service de **CQRS pragmatique**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour cQRS pragmatique.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Séparer modèles de lecture uniquement quand charge ou modèle le justifie.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Read model ciblé est versionné, relu et reproductible.
- Le parcours cQRS pragmatique possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Read model ciblé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment cQRS pragmatique se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Contract testing

But. Vérifier consumer-driven contracts et compatibilité de schémas dans la CI.

Décision à prendre

Définissez la frontière et le niveau de service de **Contract testing**. La décision doit satisfaire l'objectif de la partie: Faire évoluer les services indépendamment avec des contrats explicites et une cohérence métier maîtrisée. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour contract testing.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Vérifier consumer-driven contracts et compatibilité de schémas dans la CI.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Pipeline de contrats est versionné, relu et reproductible.
- Le parcours contract testing possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Pipeline de contrats. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment contract testing se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Kafka, Outbox, Saga et traitement fiable

Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités.

Principes directeurs

- Transactional Outbox au lieu du dual write
- Consommateurs idempotents
- Clé de partition alignée sur l'agrégat

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Exactly-once supposé de bout en bout	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Événements sans version	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
DLq utilisée comme oubliette	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Taxonomie des messages

But. Distinguer domain event, integration event, command et notification technique.

Décision à prendre

Définissez la frontière et le niveau de service de **Taxonomie des messages**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour taxonomie des messages.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Distinguer domain event, integration event, command et notification technique.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Catalogue des événements est versionné, relu et reproductible.
- Le parcours taxonomie des messages possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Catalogue des événements. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment taxonomie des messages se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Conception des topics

But. Définir nommage, partitions, réplication, rétention, compactage et ownership.

Décision à prendre

Définissez la frontière et le niveau de service de **Conception des topics**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour conception des topics.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir nommage, partitions, réplication, rétention, compactage et ownership.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Plan Kafka est versionné, relu et reproductible.
- Le parcours conception des topics possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Plan Kafka. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment conception des topics se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Transactional Outbox

But. Écrire état et événement dans la même transaction puis publier via CDC Debezium.

Décision à prendre

Définissez la frontière et le niveau de service de **Transactional Outbox**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour transactional outbox.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Écrire état et événement dans la même transaction puis publier via CDC Debezium.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
CREATE TABLE outbox_event (
  event_id uuid PRIMARY KEY,
  tenant_id uuid NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id uuid NOT NULL,
  event_type text NOT NULL,
  event_version integer NOT NULL,
  payload jsonb NOT NULL,
  occurred_at timestamptz NOT NULL
);
```

Critères de sortie

- Outbox opérationnelle est versionné, relu et reproductible.
- Le parcours transactional outbox possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Outbox opérationnelle. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment transactional outbox se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Consommateur idempotent

But. Dédupliquer par event_id dans la transaction qui applique l'effet métier.

Décision à prendre

Définissez la frontière et le niveau de service de **Consommateur idempotent**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour consommateur idempotent.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Dédupliquer par event_id dans la transaction qui applique l'effet métier.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
@Transactional
@KafkaListener(topics = "order.events.v1")
void consume(OrderEvent event) {
    if (inbox.tryInsert(event.eventId(), event.tenantId())) return;
    projection.apply(event);
}
// inbox.event_id possède une contrainte UNIQUE; l'offset Kafka est validé
// seulement après le commit de la transaction locale.
```

Critères de sortie

- Inbox/dedup store est versionné, relu et reproductible.
- Le parcours consommateur idempotent possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Inbox/dedup store. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment consommateur idempotent se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Saga de commande

But. Orchestrer réservation, approbation, paiement et compensation avec états persistés.

Décision à prendre

Définissez la frontière et le niveau de service de **Saga de commande**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour saga de commande.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Orchestrer réservation, approbation, paiement et compensation avec états persistés.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- State machine de saga est versionné, relu et reproductible.
- Le parcours saga de commande possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

State machine de saga. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment saga de commande se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Évolution des schémas

But. Appliquer compatibilité backward, métadonnées envelope et tests de registre.

Décision à prendre

Définissez la frontière et le niveau de service de **Évolution des schémas**. La décision doit satisfaire l'objectif de la partie: Construire des flux événementiels at-least-once tolérant doublons, réordonnancement et indisponibilités. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour évolution des schémas.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Appliquer compatibilité backward, métadonnées envelope et tests de registre.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Gouvernance de schémas est versionné, relu et reproductible.
- Le parcours évolution des schémas possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Gouvernance de schémas. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment évolution des schémas se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 08

Sécurité OAuth2/OIDC et Zero Trust

Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius.

Principes directeurs

- Authorization Code + PKCE pour les clients publics
- Tokens courts et audiences minimales
- Le backend revalide toujours les permissions

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Tokens dans localStorage	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Realm par tenant sans modèle opérationnel	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Secrets statiques longue durée	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Modèle Keycloak

But. Comparer realm partagé, organisations et realms dédiés selon isolation, SSO et coût.

Décision à prendre

Définissez la frontière et le niveau de service de **Modèle Keycloak**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour modèle keycloak.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Comparer realm partagé, organisations et realms dédiés selon isolation, SSO et coût.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- ADR identité multi-tenant est versionné, relu et reproductible.
- Le parcours modèle keycloak possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

ADR identité multi-tenant. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment modèle keycloak se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Angular OIDC

But. Utiliser PKCE, mémoire/session contrôlée, renouvellement sûr et garde de routes non autoritative.

Décision à prendre

Définissez la frontière et le niveau de service de **Angular OIDC**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour angular oidc.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Utiliser PKCE, mémoire/session contrôlée, renouvellement sûr et garde de routes non autoritative.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Flux web sécurisé est versionné, relu et reproductible.
- Le parcours angular oidc possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Flux web sécurisé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment angular oidc se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Flutter OIDC

But. Employer navigateur système, deep links, PKCE et stockage matériel des refresh tokens.

Décision à prendre

Définissez la frontière et le niveau de service de **Flutter OIDC**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour flutter oidc.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Employer navigateur système, deep links, PKCE et stockage matériel des refresh tokens.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Flux mobile sécurisé est versionné, relu et reproductible.
- Le parcours flutter oidc possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Flux mobile sécurisé. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment flutter oidc se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

FICHE 08.4 - IMPLÉMENTATION

Spring Resource Server

But. Valider iss, aud, exp, signature, scopes et tenant; mapper autorités explicitement.

Décision à prendre

Définissez la frontière et le niveau de service de **Spring Resource Server**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour spring resource server.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Valider iss, aud, exp, signature, scopes et tenant; mapper autorités explicitement.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health/**").permitAll()
            .requestMatchers(HttpMethod.POST, "/products/**").hasAuthority("SCOPE_catalog:write")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .build();
}
```

Critères de sortie

- SecurityFilterChain est versionné, relu et reproductible.
- Le parcours spring resource server possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

SecurityFilterChain. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment spring resource server se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Service-to-service

But. Préférer workload identity, mTLS ou client credentials à secrets rotatifs.

Décision à prendre

Définissez la frontière et le niveau de service de **Service-to-service**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour service-to-service.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Préférer workload identity, mTLS ou client credentials à secrets rotatifs.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Identités workloads est versionné, relu et reproductible.
- Le parcours service-to-service possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Identités workloads. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment service-to-service se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Threat modeling

But. Modéliser STRIDE sur login, gateway, webhooks, exports, paiements et administration.

Décision à prendre

Définissez la frontière et le niveau de service de **Threat modeling**. La décision doit satisfaire l'objectif de la partie: Authentifier utilisateurs et workloads, appliquer l'autorisation au bon niveau et réduire le blast radius. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour threat modeling.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Modéliser STRIDE sur login, gateway, webhooks, exports, paiements et administration.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Registre de menaces est versionné, relu et reproductible.
- Le parcours threat modeling possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Registre de menaces. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment threat modeling se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 09

Catalogue, pricing, stock et commande B2B

Implémenter le coeur métier avec invariants explicites et frontières cohérentes.

Principes directeurs

- Product n'est pas Offer
- Le prix appliqué est snapshoté dans la commande
- Le stock est réservé, pas seulement lu

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Prix recalculé après commande	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Overselling	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Workflow d'approbation codé en if imbriqués	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Catalog Service

But. Gérer produits, variantes, catégories, médias et publication par tenant.

Décision à prendre

Définissez la frontière et le niveau de service de **Catalog Service**. La décision doit satisfaire l'objectif de la partie: Implémenter le coeur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour catalog service.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Gérer produits, variantes, catégories, médias et publication par tenant.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Vertical slice produit est versionné, relu et reproductible.
- Le parcours catalog service possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Vertical slice produit. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment catalog service se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Pricing Service

But. Gérer price lists, contrats clients, paliers, devises, taxes et périodes de validité.

Décision à prendre

Définissez la frontière et le niveau de service de **Pricing Service**. La décision doit satisfaire l'objectif de la partie: Implémenter le coeur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour pricing service.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Gérer price lists, contrats clients, paliers, devises, taxes et périodes de validité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Moteur de prix explicable est versionné, relu et reproductible.
- Le parcours pricing service possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Moteur de prix explicable. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment pricing service se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Inventory Service

But. Réserver avec optimistic locking, expiration et événements de disponibilité.

Décision à prendre

Définissez la frontière et le niveau de service de **Inventory Service**. La décision doit satisfaire l'objectif de la partie: Implémenter le coeur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour inventory service.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Réserver avec optimistic locking, expiration et événements de disponibilité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- API de réservation est versionné, relu et reproductible.
- Le parcours inventory service possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

API de réservation. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment inventory service se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Cart et checkout

But. Valider lignes, recalcul contrôlé, adresses, livraison et idempotence.

Décision à prendre

Définissez la frontière et le niveau de service de **Cart et checkout**. La décision doit satisfaire l'objectif de la partie: Implémenter le cœur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour cart et checkout.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Valider lignes, recalcul contrôlé, adresses, livraison et idempotence.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Checkout robuste est versionné, relu et reproductible.
- Le parcours cart et checkout possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Checkout robuste. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment cart et checkout se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Order aggregate

But. Snapshot des lignes, transitions, historique et séparation ordre commercial/fulfillment.

Décision à prendre

Définissez la frontière et le niveau de service de **Order aggregate**. La décision doit satisfaire l'objectif de la partie: Implémenter le coeur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour order aggregate.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Snapshot des lignes, transitions, historique et séparation ordre commercial/fulfillment.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Cycle de commande est versionné, relu et reproductible.
- Le parcours order aggregate possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Cycle de commande. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment order aggregate se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Approval workflow

But. Exprimer seuils, centres de coût, rôles et escalades sous forme de politique versionnée.

Décision à prendre

Définissez la frontière et le niveau de service de **Approval workflow**. La décision doit satisfaire l'objectif de la partie: Implémenter le coeur métier avec invariants explicites et frontières cohérentes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour approval workflow.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Exprimer seuils, centres de coût, rôles et escalades sous forme de politique versionnée.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Moteur d'approbation est versionné, relu et reproductible.
- Le parcours approval workflow possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Moteur d'approbation. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment approval workflow se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 10

Paieement, facturation et abonnement SaaS

Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit.

Principes directeurs

- Ne jamais stocker les données carte
- Webhook authentifié et idempotent
- Ledger immuable pour les mouvements financiers

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Double débit	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Webhook forgé ou rejoué	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Confusion invoice saas et invoice commerciale	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Payment abstraction

But. Créer un port PSP et anticorruption layer pour Stripe, Adyen ou autre fournisseur.

Décision à prendre

Définissez la frontière et le niveau de service de **Payment abstraction**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour payment abstraction.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Créer un port PSP et anticorruption layer pour Stripe, Adyen ou autre fournisseur.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Adaptateur de paiement est versionné, relu et reproductible.
- Le parcours payment abstraction possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Adaptateur de paiement. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment payment abstraction se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Payment intents

But. Gérer authorize/capture, asynchronisme, 3DS, échecs et idempotency provider.

Décision à prendre

Définissez la frontière et le niveau de service de **Payment intents**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour payment intents.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Gérer authorize/capture, asynchronisme, 3DS, échecs et idempotency provider.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- State machine paiement est versionné, relu et reproductible.
- Le parcours payment intents possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

State machine paiement. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment payment intents se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Webhooks

But. Vérifier signature sur corps brut, timestamp, replay, endpoint secret et journalisation sûre.

Décision à prendre

Définissez la frontière et le niveau de service de **Webhooks**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour webhooks.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Vérifier signature sur corps brut, timestamp, replay, endpoint secret et journalisation sûre.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Endpoint webhook durci est versionné, relu et reproductible.
- Le parcours webhooks possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Endpoint webhook durci. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment webhooks se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Billing B2B

But. Générer facture, avoir, numérotation, taxes et pièces d'audit selon juridiction.

Décision à prendre

Définissez la frontière et le niveau de service de **Billing B2B**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour billing b2b.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Générer facture, avoir, numérotation, taxes et pièces d'audit selon juridiction.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Cycle de facturation est versionné, relu et reproductible.
- Le parcours billing b2b possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Cycle de facturation. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment billing b2b se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Subscription SaaS

But. Modéliser plans, entitlements, metering, essais, proration et dunning.

Décision à prendre

Définissez la frontière et le niveau de service de **Subscription SaaS**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour subscription saas.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Modéliser plans, entitlements, metering, essais, proration et dunning.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Entitlement service est versionné, relu et reproductible.
- Le parcours subscription saas possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Entitlement service. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment subscription saas se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Réconciliation

But. Comparer ledger interne, PSP et banque; traiter écarts et clôture quotidienne.

Décision à prendre

Définissez la frontière et le niveau de service de **Réconciliation**. La décision doit satisfaire l'objectif de la partie: Séparer monétisation de la plateforme et paiements des commandes, avec réconciliation et audit. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour réconciliation.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Comparer ledger interne, PSP et banque; traiter écarts et clôture quotidienne.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Job de reconciliation est versionné, relu et reproductible.
- Le parcours réconciliation possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Job de reconciliation. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment réconciliation se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 11

Angular 22: architecture web professionnelle

Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals.

Principes directeurs

- Standalone et lazy loading par feature
- Etat local d'abord, store global seulement si partagé
- UI ne connaît pas les DTO backend

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Store global géant	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Subscriptions manuelles	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Guards considérés comme sécurité	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Workspace et frontières

But. Structurer core, shared UI et features catalog, orders, billing, admin.

Décision à prendre

Définissez la frontière et le niveau de service de **Workspace et frontières**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour workspace et frontières.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Structurer core, shared UI et features catalog, orders, billing, admin.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Architecture Angular est versionné, relu et reproductible.
- Le parcours workspace et frontières possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Architecture Angular. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment workspace et frontières se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Signaux et dérivations

But. Utiliser signal, computed, resource/httpResource stable et effects seulement pour side effects.

Décision à prendre

Définissez la frontière et le niveau de service de **Signaux et dérivations**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour signaux et dérivations.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Utiliser signal, computed, resource/httpResource stable et effects seulement pour side effects.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
@Injectable()
export class CatalogStore {
  private readonly api = inject(CatalogApi);
  readonly query = signal('');
  readonly products = signal<readonly Product[]>([]);
  readonly filtered = computed(() => {
    const q = this.query().trim().toLowerCase();
    return this.products().filter(p => p.name.toLowerCase().includes(q));
  });
  async reload(): Promise<void> {
    this.products.set(await firstValueFrom(this.api.list()));
  }
}
```

Critères de sortie

- State model réactif est versionné, relu et reproductible.
- Le parcours signaux et dérivations possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livable

State model réactif. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment signaux et dérivations se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Signal Forms

But. Construire formulaires typés, validations synchrones/asynchrones et mapping vers commandes.

Décision à prendre

Définissez la frontière et le niveau de service de **Signal Forms**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour signal forms.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Construire formulaires typés, validations synchrones/asynchrones et mapping vers commandes.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Formulaire produit est versionné, relu et reproductible.
- Le parcours signal forms possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Formulaire produit. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment signal forms se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

FICHE 11.4 - IMPLÉMENTATION

HTTP et erreurs

But. Intercepteurs fonctionnels pour auth, tenant, trace, retry sûr et Problem Details.

Décision à prendre

Définissez la frontière et le niveau de service de **HTTP et erreurs**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour http et erreurs.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Intercepteurs fonctionnels pour auth, tenant, trace, retry sûr et Problem Details.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const session = inject(SessionService);
  const traceId = crypto.randomUUID();
  const headers = req.headers
    .set('Authorization', `Bearer ${session.accessToken()}`)
    .set('traceparent', session.traceparent(traceId));
  return next(req.clone({headers})).pipe(
    catchError(error => throwError(() => mapProblemDetails(error)))
  );
};
```

Critères de sortie

- Client API cohérent est versionné, relu et reproductible.
- Le parcours http et erreurs possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livable

Client API cohérent. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment http et erreurs se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Routing et autorisation UX

But. Lazy routes, resolvers ciblés, canMatch et masquage de capacités selon entitlements.

Décision à prendre

Définissez la frontière et le niveau de service de **Routing et autorisation UX**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour routing et autorisation ux.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Lazy routes, resolvers ciblés, canMatch et masquage de capacités selon entitlements.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Navigation par rôle est versionné, relu et reproductible.
- Le parcours routing et autorisation ux possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Navigation par rôle. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment routing et autorisation ux se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Performance et accessibilité

But. Zoneless, OnPush, defer, budgets, SSR si utile, WCAG 2.2 AA et i18n.

Décision à prendre

Définissez la frontière et le niveau de service de **Performance et accessibilité**. La décision doit satisfaire l'objectif de la partie: Construire un portail B2B rapide, accessible et testable avec une architecture feature-first à base de Signals. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour performance et accessibilité.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Zoneless, OnPush, defer, budgets, SSR si utile, WCAG 2.2 AA et i18n.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Budget qualité web est versionné, relu et reproductible.
- Le parcours performance et accessibilité possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Budget qualité web. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment performance et accessibilité se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Flutter 3.47: application mobile cohérente

Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular.

Principes directeurs

- Feature-first et dépendances vers le domaine
- Riverpod/BLoC choisi et standardisé, pas mélangé
- Offline explicite avec source de vérité

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Token dans sharedpreferences	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Dto utilisés jusque dans les widgets	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Synchronisation offline sans conflits	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Structure clean architecture

But. Organiser core, features/domain, application, data et presentation avec règles d'import.

Décision à prendre

Définissez la frontière et le niveau de service de **Structure clean architecture**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour structure clean architecture.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Organiser core, features/domain, application, data et presentation avec règles d'import.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
lib/
  core/                # erreurs, réseau, sécurité, design system
  features/catalog/
    domain/            # Product, repositories abstraits
    application/       # use cases et états
    data/              # DTO, API, cache, mappers
    presentation/      # pages, widgets, providers
  features/orders/
  app/                # router, bootstrap, composition root
```

Critères de sortie

- Architecture Flutter est versionné, relu et reproductible.
- Le parcours structure clean architecture possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Architecture Flutter. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment structure clean architecture se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Navigation

But. Configurer routes typées, deep links OIDC, guards UX et restauration d'état.

Décision à prendre

Définissez la frontière et le niveau de service de **Navigation**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour navigation.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Configurer routes typées, deep links OIDC, guards UX et restauration d'état.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Router mobile est versionné, relu et reproductible.
- Le parcours navigation possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Router mobile. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment navigation se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

FICHE 12.3 - IMPLÉMENTATION

Client HTTP

But. Intercepter auth, tenant et trace; classier erreurs et appliquer retries idempotents.

Décision à prendre

Définissez la frontière et le niveau de service de **Client HTTP**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour client http.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Intercepter auth, tenant et trace; classier erreurs et appliquer retries idempotents.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
final class TenantAuthInterceptor extends Interceptor {
  TenantAuthInterceptor(this.session);
  final Session session;
  @override
  void onRequest(RequestOptions o, RequestInterceptorHandler h) {
    o.headers['Authorization'] = 'Bearer ${session.accessToken}';
    o.headers['traceparent'] = session.newTraceparent();
    h.next(o);
  }
}
```

Critères de sortie

- API client mobile est versionné, relu et reproductible.
- Le parcours client http possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

API client mobile. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment client http se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

State management

But. Modéliser loading/data/error, commandes, annulation et rafraîchissement sans logique dans widgets.

Décision à prendre

Définissez la frontière et le niveau de service de **State management**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour state management.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Modéliser loading/data/error, commandes, annulation et rafraîchissement sans logique dans widgets.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- State model uniforme est versionné, relu et reproductible.
- Le parcours state management possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

State model uniforme. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment state management se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Offline first

But. Cache chiffré, outbox locale, synchronisation, conflits et affichage de fraîcheur.

Décision à prendre

Définissez la frontière et le niveau de service de **Offline first**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour offline first.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Cache chiffré, outbox locale, synchronisation, conflits et affichage de fraîcheur.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Stratégie offline est versionné, relu et reproductible.
- Le parcours offline first possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Stratégie offline. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment offline first se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Qualité mobile

But. Golden tests, widget tests, intégration, performance Impeller, accessibilité et release signing.

Décision à prendre

Définissez la frontière et le niveau de service de **Qualité mobile**. La décision doit satisfaire l'objectif de la partie: Répliquer les mêmes frontières métier et contrats sans copier l'implémentation Angular. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour qualité mobile.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Golden tests, widget tests, intégration, performance Impeller, accessibilité et release signing.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Pipeline mobile est versionné, relu et reproductible.
- Le parcours qualité mobile possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Pipeline mobile. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment qualité mobile se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests et stratégie de qualité

Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques.

Principes directeurs

- Tester les invariants au niveau le moins coûteux
- Testcontainers pour les comportements réels
- Peu d'E2E, mais sur les revenus et la sécurité

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Mocks qui confirment l'implémentation	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Tests flaky	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Couverture utilisée comme objectif unique	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Tests de domaine

But. Tester agrégats et value objects sans contexte Spring, horloge réelle ni UUID aléatoire.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests de domaine**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests de domaine.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Tester agrégats et value objects sans contexte Spring, horloge réelle ni UUID aléatoire.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Suite unitaire déterministe est versionné, relu et reproductible.
- Le parcours tests de domaine possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Suite unitaire déterministe. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests de domaine se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests d'application

But. Tester ports, autorisations métier, transactions et publication d'événements avec fakes.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests d'application**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests d'application.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Tester ports, autorisations métier, transactions et publication d'événements avec fakes.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Tests de cas d'usage est versionné, relu et reproductible.
- Le parcours tests d'application possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Tests de cas d'usage. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests d'application se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests d'intégration

But. Lancer PostgreSQL, Kafka et Keycloak avec Testcontainers et données minimales.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests d'intégration**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests d'intégration.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Lancer PostgreSQL, Kafka et Keycloak avec Testcontainers et données minimales.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Harness d'intégration est versionné, relu et reproductible.
- Le parcours tests d'intégration possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Harness d'intégration. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests d'intégration se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests de contrat

But. Valider OpenAPI, consumer contracts et schémas événementiels dans chaque PR.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests de contrat**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests de contrat.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Valider OpenAPI, consumer contracts et schémas événementiels dans chaque PR.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Quality gate contrats est versionné, relu et reproductible.
- Le parcours tests de contrat possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Quality gate contrats. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests de contrat se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Tests frontends

But. Vitest/Testing Library pour Angular; unit/widget/golden pour Flutter.

Décision à prendre

Définissez la frontière et le niveau de service de **Tests frontends**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour tests frontends.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Vitest/Testing Library pour Angular; unit/widget/golden pour Flutter.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Pyramides frontend est versionné, relu et reproductible.
- Le parcours tests frontends possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Pyramides frontend. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment tests frontends se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

E2E et performance

But. Couvrir onboarding, achat, approbation, paiement; tester k6 avec seuils p95 et erreurs.

Décision à prendre

Définissez la frontière et le niveau de service de **E2E et performance**. La décision doit satisfaire l'objectif de la partie: Créer une pyramide de tests rapide qui sécurise domaine, intégrations, contrats et parcours critiques. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour e2e et performance.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Couvrir onboarding, achat, approbation, paiement; tester k6 avec seuils p95 et erreurs.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Suite release est versionné, relu et reproductible.
- Le parcours e2e et performance possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Suite release. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment e2e et performance se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 14

Résilience et performance distribuée

Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance.

Principes directeurs

- Timeout partout, retry rarement
- Budget de latence de bout en bout
- Backpressure avant autoscaling

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Retry storm	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Fallback mensonger	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Pool de connexions surdimensionné	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Timeout budgets

But. Décomposer le p95 cible entre gateway, service, base, cache et appels externes.

Décision à prendre

Définissez la frontière et le niveau de service de **Timeout budgets**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour timeout budgets.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Décomposer le p95 cible entre gateway, service, base, cache et appels externes.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Budget de latence est versionné, relu et reproductible.
- Le parcours timeout budgets possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Budget de latence. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment timeout budgets se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Retry et backoff

But. Retry seulement transient et idempotent, jitter, limite d'essais et retry budget.

Décision à prendre

Définissez la frontière et le niveau de service de **Retry et backoff**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour retry et backoff.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Retry seulement transient et idempotent, jitter, limite d'essais et retry budget.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Politique de retry est versionné, relu et reproductible.
- Le parcours retry et backoff possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Politique de retry. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment retry et backoff se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Circuit breaker

But. Configurer fenêtres, seuils et transitions à partir de mesures, non de valeurs copiées.

Décision à prendre

Définissez la frontière et le niveau de service de **Circuit breaker**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour circuit breaker.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Configurer fenêtres, seuils et transitions à partir de mesures, non de valeurs copiées.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Protection des dépendances est versionné, relu et reproductible.
- Le parcours circuit breaker possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Protection des dépendances. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment circuit breaker se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Bulkheads

But. Séparer pools et quotas pour endpoints, tenants et intégrations lentes.

Décision à prendre

Définissez la frontière et le niveau de service de **Bulkheads**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour bulkheads.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Séparer pools et quotas pour endpoints, tenants et intégrations lentes.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Plan d'isolation est versionné, relu et reproductible.
- Le parcours bulkheads possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Plan d'isolation. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment bulkheads se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Caching

But. Définir ownership, TTL, invalidation événementielle, stampede protection et stale-while-revalidate.

Décision à prendre

Définissez la frontière et le niveau de service de **Caching**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour caching.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir ownership, TTL, invalidation événementielle, stampede protection et stale-while-revalidate.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Stratégie cache est versionné, relu et reproductible.
- Le parcours caching possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Stratégie cache. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment caching se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Chaos engineering

But. Tester pod kill, latence DB, broker indisponible et PSP lent avec hypothèse et rollback.

Décision à prendre

Définissez la frontière et le niveau de service de **Chaos engineering**. La décision doit satisfaire l'objectif de la partie: Concevoir pour les pannes partielles, contrôler les ressources et prouver les objectifs de performance. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour chaos engineering.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Tester pod kill, latence DB, broker indisponible et PSP lent avec hypothèse et rollback.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Expériences de chaos est versionné, relu et reproductible.
- Le parcours chaos engineering possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Expériences de chaos. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment chaos engineering se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Observabilité, SRE et opérations

Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition.

Principes directeurs

- Logs, métriques et traces corrélés
- Alertes sur symptômes utilisateur
- Runbook attaché à chaque alerte

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Tenant_id en label haute cardinalité	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Logs contenant pii	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Dashboards sans action	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

OpenTelemetry

But. Propager traceparent/baggage via HTTP et Kafka, exporter par collector et échantillonner.

Décision à prendre

Définissez la frontière et le niveau de service de **OpenTelemetry**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour opentelemetry.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Propager traceparent/baggage via HTTP et Kafka, exporter par collector et échantillonner.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Tracing end-to-end est versionné, relu et reproductible.
- Le parcours opentelemetry possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Tracing end-to-end. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment opentelemetry se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Logs structurés

But. JSON, trace_id, service, version et codes stables; masquage secrets et données personnelles.

Décision à prendre

Définissez la frontière et le niveau de service de **Logs structurés**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour logs structurés.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: JSON, trace_id, service, version et codes stables; masquage secrets et données personnelles.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Contrat de logs est versionné, relu et reproductible.
- Le parcours logs structurés possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Contrat de logs. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment logs structurés se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Métriques RED/USE

But. Mesurer rate, errors, duration et saturation avec cardinalité bornée.

Décision à prendre

Définissez la frontière et le niveau de service de **Métriques RED/USE**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour métriques red/use.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Mesurer rate, errors, duration et saturation avec cardinalité bornée.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Dashboard golden signals est versionné, relu et reproductible.
- Le parcours métriques red/use possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Dashboard golden signals. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment métriques red/use se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

FICHE 15.4 - IMPLÉMENTATION

SLI/SLO

But. Définir disponibilité et latence par parcours; calculer error budget sur 28 ou 30 jours.

Décision à prendre

Définissez la frontière et le niveau de service de **SLI/SLO**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour sli/slo.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Définir disponibilité et latence par parcours; calculer error budget sur 28 ou 30 jours.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
service: checkout
window: 30d
objectives:
  availability:
    target: 99.98
    good: "status < 500 and checkout_result != technical_failure"
  latency:
    target: 95.00
    threshold_ms: 800
alerts:
  - burn_rate: 14.4
    short_window: 5m
    long_window: 1h
```

Critères de sortie

- Catalogue SLO est versionné, relu et reproductible.
- Le parcours sli/slo possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livable

Catalogue SLO. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment sli/slo se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Alerting

But. Multi-window burn-rate, déduplication, ownership et escalade adaptée à la gravité.

Décision à prendre

Définissez la frontière et le niveau de service de **Alerting**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour alerting.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Multi-window burn-rate, déduplication, ownership et escalade adaptée à la gravité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Alertes actionnables est versionné, relu et reproductible.
- Le parcours alerting possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Alertes actionnables. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment alerting se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Incident management

But. Préparer runbooks, rôles incident, communication, timeline et postmortem sans blâme.

Décision à prendre

Définissez la frontière et le niveau de service de **Incident management**. La décision doit satisfaire l'objectif de la partie: Rendre chaque parcours traçable et piloter la fiabilité par SLO plutôt que par intuition. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour incident management.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Préparer runbooks, rôles incident, communication, timeline et postmortem sans blâme.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Kit d'incident est versionné, relu et reproductible.
- Le parcours incident management possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Kit d'incident. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment incident management se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 16

Docker, Kubernetes et sécurité supply chain

Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes.

Principes directeurs

- Image immuable par digest
- Least privilege à chaque couche
- Probes différentes pour vie et disponibilité

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Latest en production	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Secret dans image ou manifest	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Liveness qui amplifie une panne	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Images multi-stage

But. Compiler séparément, runtime minimal, utilisateur non root, read-only filesystem et SBOM.

Décision à prendre

Définissez la frontière et le niveau de service de **Images multi-stage**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour images multi-stage.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Compiler séparément, runtime minimal, utilisateur non root, read-only filesystem et SBOM.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
FROM maven:3.9-eclipse-temurin-25 AS build
WORKDIR /workspace
COPY pom.xml .
COPY src src
RUN mvn -B -DskipTests package

FROM eclipse-temurin:25-jre
RUN useradd --system --uid 10001 app
USER 10001
COPY --from=build /workspace/target/app.jar /app.jar
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

Critères de sortie

- Dockerfiles durcis est versionné, relu et reproductible.
- Le parcours images multi-stage possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livable

Dockerfiles durcis. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment images multi-stage se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Déploiements Kubernetes

But. Requests/limits, rollout progressif, PDB, topology spread et termination grace.

Décision à prendre

Définissez la frontière et le niveau de service de **Déploiements Kubernetes**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour déploiements kubernetes.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Requests/limits, rollout progressif, PDB, topology spread et termination grace.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  template:
    spec:
      securityContext: {runAsNonRoot: true}
      containers:
        - name: catalog
          image: registry/catalog@sha256:IMMUTABLE_DIGEST
          resources:
            requests: {cpu: 250m, memory: 384Mi}
            limits: {memory: 768Mi}
          readinessProbe:
            httpGet: {path: /actuator/health/readiness, port: 8080}
```

Critères de sortie

- Deployment production est versionné, relu et reproductible.
- Le parcours déploiements kubernetes possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Deployment production. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment déploiements kubernetes se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Probes

But. Liveness locale, readiness incluant capacité de servir et startup adaptée au JVM warmup.

Décision à prendre

Définissez la frontière et le niveau de service de **Probes**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour probes.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Liveness locale, readiness incluant capacité de servir et startup adaptée au JVM warmup.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Politique de probes est versionné, relu et reproductible.
- Le parcours probes possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Politique de probes. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment probes se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Autoscaling

But. HPA sur métriques pertinentes, VPA en recommandation et cluster autoscaler avec capacité tampon.

Décision à prendre

Définissez la frontière et le niveau de service de **Autoscaling**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour autoscaling.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: HPA sur métriques pertinentes, VPA en recommandation et cluster autoscaler avec capacité tampon.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Plan de scaling est versionné, relu et reproductible.
- Le parcours autoscaling possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Plan de scaling. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment autoscaling se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Network policies

But. Default deny, flux explicitement autorisés, egress contrôlé et DNS pris en compte.

Décision à prendre

Définissez la frontière et le niveau de service de **Network policies**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour network policies.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Default deny, flux explicitement autorisés, egress contrôlé et DNS pris en compte.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Matrice réseau est versionné, relu et reproductible.
- Le parcours network policies possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Matrice réseau. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment network policies se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Supply chain

But. SBOM CycloneDX, scan CVE, signature Cosign, provenance SLSA et admission policy.

Décision à prendre

Définissez la frontière et le niveau de service de **Supply chain**. La décision doit satisfaire l'objectif de la partie: Construire et exécuter des artefacts minimaux, non privilégiés et vérifiables sur Kubernetes. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour supply chain.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: SBOM CycloneDX, scan CVE, signature Cosign, provenance SLSA et admission policy.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Chaîne de confiance est versionné, relu et reproductible.
- Le parcours supply chain possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Chaîne de confiance. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment supply chain se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 17

CI/CD, GitOps et Infrastructure as Code

Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé.

Principes directeurs

- Build once, promote many
- Git décrit l'état désiré
- Les secrets ne transitent jamais dans les logs ou le state

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Rebuild différent par environnement	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Actions non épinglées	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Auto-sync sans garde-fous	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Pipeline backend

But. Compile, tests, architecture, SAST, SCA, SBOM, image, signature et publication.

Décision à prendre

Définissez la frontière et le niveau de service de **Pipeline backend**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour pipeline backend.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Compile, tests, architecture, SAST, SCA, SBOM, image, signature et publication.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- CI Spring Boot est versionné, relu et reproductible.
- Le parcours pipeline backend possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

CI Spring Boot. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment pipeline backend se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Pipeline Angular

But. Lint, typecheck, Vitest, a11y, build budgets, E2E critique et image edge.

Décision à prendre

Définissez la frontière et le niveau de service de **Pipeline Angular**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour pipeline angular.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Lint, typecheck, Vitest, a11y, build budgets, E2E critique et image edge.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- CI Angular est versionné, relu et reproductible.
- Le parcours pipeline angular possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

CI Angular. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment pipeline angular se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Pipeline Flutter

But. Analyze, tests, build signé, distribution interne et promotion stores.

Décision à prendre

Définissez la frontière et le niveau de service de **Pipeline Flutter**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour pipeline flutter.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Analyze, tests, build signé, distribution interne et promotion stores.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- CI mobile est versionné, relu et reproductible.
- Le parcours pipeline flutter possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

CI mobile. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment pipeline flutter se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Terraform

But. Modules versionnés, remote state chiffré, plan en PR, apply protégé et policy as code.

Décision à prendre

Définissez la frontière et le niveau de service de **Terraform**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour terraform.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Modules versionnés, remote state chiffré, plan en PR, apply protégé et policy as code.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
terraform {
  required_version = "~> 1.0"
  backend "s3" { encrypt = true }
}
module "platform" {
  source      = "git::https://example.invalid/platform.git//cluster?ref=v1.4.0"
  environment = var.environment
  tags        = local.mandatory_tags
}
```

Critères de sortie

- Socle IaC est versionné, relu et reproductible.
- Le parcours terraform possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Socle IaC. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment terraform se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Argo CD

But. Applications par environnement, sync waves, health checks, progressive delivery et rollback Git.

Décision à prendre

Définissez la frontière et le niveau de service de **Argo CD**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour argo cd.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Applications par environnement, sync waves, health checks, progressive delivery et rollback Git.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- GitOps opérationnel est versionné, relu et reproductible.
- Le parcours argo cd possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

GitOps opérationnel. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment argo cd se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Gestion des secrets

But. External Secrets/Vault, workload identity, rotation et audit sans secret Git.

Décision à prendre

Définissez la frontière et le niveau de service de **Gestion des secrets**. La décision doit satisfaire l'objectif de la partie: Automatiser une livraison reproductible avec promotion d'artefacts, contrôles et rollback prouvé. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour gestion des secrets.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: External Secrets/Vault, workload identity, rotation et audit sans secret Git.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Chaîne de secrets est versionné, relu et reproductible.
- Le parcours gestion des secrets possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Chaîne de secrets. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment gestion des secrets se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

PARTIE 18

Gouvernance, coûts et évolution d'architecture

Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût.

Principes directeurs

- Golden paths plutôt que règles manuelles
- ADR courts et vivants
- FinOps relié aux décisions produit

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Plateforme interne prématurée	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Standard unique pour tous les cas	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Coûts sans attribution	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Architecture Decision Records

But. Documenter contexte, options, décision, conséquences, date de revue et signaux de réversibilité.

Décision à prendre

Définissez la frontière et le niveau de service de **Architecture Decision Records**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour architecture decision records.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Documenter contexte, options, décision, conséquences, date de revue et signaux de réversibilité.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Extrait de référence

```
# ADR-007 - Shared schema + PostgreSQL RLS
Status: Accepted
Date de revue: 2027-02-01
Contexte: 10 000 tenants, coût initial constraint, isolation obligatoire.
Décision: shared schema, tenant_id, FORCE RLS, rôle applicatif sans BYPASSRLS.
Conséquences: tests anti-fuite obligatoires; possibilité de déplacer les tenants
entreprise vers une base dédiée.
Signaux de révision: noisy neighbor, contraintes réglementaires, taille > seuil.
```

Critères de sortie

- Journal ADR est versionné, relu et reproductible.
- Le parcours architecture decision records possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Journal ADR. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment architecture decision records se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

C4 et documentation

But. Maintenir System Context, Container, Component ciblé et diagrammes de séquence critiques.

Décision à prendre

Définissez la frontière et le niveau de service de **C4 et documentation**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour c4 et documentation.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Maintenir System Context, Container, Component ciblé et diagrammes de séquence critiques.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Documentation as code est versionné, relu et reproductible.
- Le parcours c4 et documentation possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Documentation as code. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment c4 et documentation se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Team Topologies

But. Aligner stream-aligned teams, platform team et interactions sur les contextes.

Décision à prendre

Définissez la frontière et le niveau de service de **Team Topologies**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour team topologies.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Aligner stream-aligned teams, platform team et interactions sur les contextes.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Carte des responsabilités est versionné, relu et reproductible.
- Le parcours team topologies possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Carte des responsabilités. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment team topologies se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Golden paths

But. Créer templates, pipelines et observabilité par défaut tout en permettant l'exception argumentée.

Décision à prendre

Définissez la frontière et le niveau de service de **Golden paths**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour golden paths.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Créer templates, pipelines et observabilité par défaut tout en permettant l'exception argumentée.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Développer platform minimale est versionné, relu et reproductible.
- Le parcours golden paths possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Développer platform minimale. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment golden paths se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

FinOps

But. Tagger coûts, mesurer coût par tenant/commande, rightsizing et capacité réservée.

Décision à prendre

Définissez la frontière et le niveau de service de **FinOps**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour finops.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Tagger coûts, mesurer coût par tenant/commande, rightsizing et capacité réservée.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Tableau de coût unitaire est versionné, relu et reproductible.
- Le parcours finops possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Tableau de coût unitaire. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment finops se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Strangler et évolution

But. Fusionner ou scinder services selon métriques de changement, charge et ownership.

Décision à prendre

Définissez la frontière et le niveau de service de **Strangler et évolution**. La décision doit satisfaire l'objectif de la partie: Maintenir la cohérence sans comité bloquant et arbitrer fiabilité, vitesse, complexité et coût. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour strangler et évolution.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Fusionner ou scinder services selon métriques de changement, charge et ownership.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Plan d'évolution est versionné, relu et reproductible.
- Le parcours strangler et évolution possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Plan d'évolution. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment strangler et évolution se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Capstone: trajectoire d'implémentation complète

Assembler les décisions en une exécution progressive, démontrable et exploitable en production.

Principes directeurs

- Chaque phase livre un parcours vertical
- Definition of Done inclut exploitation et sécurité
- Les risques majeurs sont testés tôt

Risques à réduire

Risque	Signal précoce	Réponse d'architecte
Construction horizontale sans valeur	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Big bang de production	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir
Absence de critères de sortie	Couplage, incident, délai ou coût augmente	Mesurer, limiter le blast radius, décider et revoir

Architecture review - questions

- Quelle hypothèse métier rend cette complexité nécessaire aujourd'hui?
- Quelle panne ou menace est contenue, et quel est le blast radius restant?
- Comment prouver le comportement par test, métrique, trace ou exercice opérationnel?
- Quelle alternative plus simple a été étudiée et pourquoi est-elle insuffisante?
- À quel signal et à quelle date cette décision doit-elle être réévaluée?

Phase 0 - Discovery

But. Event Storming, QAS, threat model, spikes RLS/Kafka/OIDC et backlog de risques.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 0 - Discovery**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 0 - discovery.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Event Storming, QAS, threat model, spikes RLS/Kafka/OIDC et backlog de risques.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Dossier d'architecture initial est versionné, relu et reproductible.
- Le parcours phase 0 - discovery possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Dossier d'architecture initial. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 0 - discovery se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Phase 1 - Walking skeleton

But. Login, tenant, produit minimal, commande vide, traces et déploiement GitOps.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 1 - Walking skeleton**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 1 - walking skeleton.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Login, tenant, produit minimal, commande vide, traces et déploiement GitOps.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Système traversant est versionné, relu et reproductible.
- Le parcours phase 1 - walking skeleton possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Système traversant. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 1 - walking skeleton se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Phase 2 - Commerce

But. Catalogue, pricing, stock, checkout, approbation et saga sans paiement réel.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 2 - Commerce**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 2 - commerce.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Catalogue, pricing, stock, checkout, approbation et saga sans paiement réel.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- MVP B2B est versionné, relu et reproductible.
- Le parcours phase 2 - commerce possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

MVP B2B. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 2 - commerce se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Phase 3 - Revenue

But. Paiement sandbox, facturation, subscription SaaS, webhooks et reconciliation.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 3 - Revenue**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 3 - revenue.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Paiement sandbox, facturation, subscription SaaS, webhooks et reconciliation.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Flux de revenus est versionné, relu et reproductible.
- Le parcours phase 3 - revenue possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Flux de revenus. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 3 - revenue se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Phase 4 - Scale

But. Tests charge, SLO, autoscaling, chaos, DR, sécurité supply chain et coût unitaire.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 4 - Scale**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 4 - scale.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Tests charge, SLO, autoscaling, chaos, DR, sécurité supply chain et coût unitaire.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Release candidate est versionné, relu et reproductible.
- Le parcours phase 4 - scale possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Release candidate. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 4 - scale se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Phase 5 - Architecture review

But. Évaluer preuves, exceptions, dette, roadmap et aptitude à opérer.

Décision à prendre

Définissez la frontière et le niveau de service de **Phase 5 - Architecture review**. La décision doit satisfaire l'objectif de la partie: Assembler les décisions en une exécution progressive, démontrable et exploitable en production. N'acceptez une dépendance ou une abstraction que si elle réduit un risque identifié ou accélère plusieurs parcours réels.

Plan d'implémentation

- Écrire le scénario nominal et au moins trois scénarios d'échec pour phase 5 - architecture review.
- Définir le contrat entrant, le résultat, les erreurs stables et les données sensibles; focus: Évaluer preuves, exceptions, dette, roadmap et aptitude à opérer.
- Identifier propriétaire métier, propriétaire technique, dépendances synchrones et événements consommés/publiés.
- Implémenter un vertical slice minimal avec logs corrélés, métriques RED et trace distribuée.
- Ajouter tests de domaine, intégration réelle, contrat et cas négatif cross-tenant lorsque des données sont concernées.
- Mesurer latence, débit, saturation, coût et comportement lors d'une dépendance indisponible.
- Enregistrer la décision dans un ADR si elle modifie une frontière, un risque, un coût récurrent ou un attribut de qualité.

Critères de sortie

- Dossier de soutenance architecte est versionné, relu et reproductible.
- Le parcours phase 5 - architecture review possède critères d'acceptation métier et non fonctionnels.
- Aucun secret, donnée personnelle ou identifiant de tenant non borné n'apparaît dans les logs.
- Les erreurs partielles et doublons ont un comportement déterministe documenté.
- Un dashboard ou rapport permet de vérifier le résultat sans inspection manuelle de la base.

Livrable

Dossier de soutenance architecte. Ajoutez une courte note indiquant l'owner, la commande de vérification, les métriques attendues, la procédure de rollback et la prochaine date de revue.

Défi architecte

Soutenez ce choix en cinq minutes: contexte, deux alternatives, compromis, preuve et condition de réversibilité. Puis expliquez comment phase 5 - architecture review se comporte si PostgreSQL, Kafka, Keycloak ou un fournisseur externe devient lent pendant dix minutes.

Annexe A - Structure recommandée du futur dépôt

À créer plus tard dans un nouveau dossier, pas dans le projet actuel.

```
saas-b2b/  
  apps/  
    web-angular/  
    mobile-flutter/  
  services/  
    gateway/  
    tenant-service/  
    catalog-service/  
    pricing-service/  
    inventory-service/  
    order-service/  
    payment-service/  
    billing-service/  
    notification-service/  
  contracts/  
    openapi/  
    events/  
  platform/  
    terraform/  
    kubernetes/  
    argocd/  
    observability/  
  docs/  
    adr/  
    c4/  
    runbooks/  
  tests/  
    e2e/  
    performance/  
    chaos/
```

Règles de dépendance

- Chaque service est buildable et déployable seul.
- contracts contient des contrats publiés, jamais de modèle métier Java partagé.
- platform fournit des golden paths; un domaine ne dépend pas du code d'un autre domaine.
- Les frontends génèrent ou adaptent leurs clients depuis OpenAPI, puis mappent les DTO vers leur modèle.
- Les environnements promeuvent le même digest d'image.

Annexe B - ADR obligatoires

- ADR-001 Frontières des bounded contexts
- ADR-002 Topologie microservices
- ADR-003 Shared schema + RLS et voie enterprise
- ADR-004 Modèle d'identité Keycloak
- ADR-005 REST, événements et règles de synchronisme
- ADR-006 Outbox/CDC et idempotence
- ADR-007 Saga commande
- ADR-008 Stratégie Angular Signals
- ADR-009 State management et offline Flutter
- ADR-010 Plateforme Kubernetes et GitOps
- ADR-011 Observabilité et SLO
- ADR-012 Disaster recovery et rétention

Template

```
# ADR-NNN - Titre
Status: Proposed | Accepted | Superseded
Contexte et forces en tension:
Options considérées:
Décision et justification:
Conséquences positives et négatives:
Preuves / fitness functions:
Plan de migration et rollback:
Owner, date de revue, signaux de réévaluation:
```

Annexe C - Checklist sécurité

- OIDC Authorization Code + PKCE; aucun mot de passe traité par Angular/Flutter
- Validation iss, aud, exp, signature, scopes et tenant dans chaque resource server
- Tokens absents des logs, URLs et crash reports
- RLS forcée, rôle applicatif sans BYPASSRLS, tests cross-tenant
- CORS restrictif; CSP, HSTS et cookies adaptés si BFF
- Rate limits par tenant et par identité, limites de taille et timeouts
- Webhooks: signature, corps brut, timestamp, anti-replay et idempotence
- Secrets via workload identity/Vault; rotation et révocation testées
- Images non root, read-only, capabilities supprimées, network default deny
- SAST, SCA, SBOM, scan image, signature et provenance
- Exports et sauvegardes chiffrés; suppression et rétention GDPR
- Threat model revu à chaque nouveau trust boundary

Critère de passage

Une fonctionnalité de revenu ou d'administration ne passe pas en production sans cas négatifs d'autorisation, preuve d'isolation tenant, audit event et procédure de révocation.

Annexe D - Checklist production readiness

- Owner et astreinte définis
- SLO, dashboard RED et alertes burn-rate
- Runbook pour saturation, dépendance lente, backlog Kafka et erreur DB
- Requests/limits, HPA et PodDisruptionBudget
- Readiness/liveness/startup probes validées
- Migrations expand/contract et rollback applicatif
- Sauvegarde restaurée dans un environnement isolé
- RPO/RTO testés par exercice
- Feature flags avec expiration et owner
- Dégradation contrôlée et files bornées
- Canary ou progressive delivery
- Coût par transaction et capacité de pointe connus
- Audit security et dépendances critiques à jour
- Post-déploiement avec métriques et rollback automatique

Annexe E - Plan de progression sur 24 semaines

Semaines	Résultat
1-2	Discovery, Event Storming, QAS, threat model
3-4	Walking skeleton: OIDC, tenant, traces, GitOps
5-7	Catalog + Pricing + Angular vertical slice
8-10	Inventory + Order + saga initiale
11-12	Flutter catalogue et approbation
13-14	Payment sandbox + webhooks + ledger
15-16	Billing + subscription + entitlements
17-18	Contract tests, performance, sécurité
19-20	SLO, chaos, runbooks, incident game day
21-22	Terraform, supply chain, DR
23	FinOps, architecture review, dette
24	Soutenance: démo, panne simulée, décisions et roadmap

Rituel hebdomadaire

- Une décision d'architecture soutenue avec alternatives
- Un vertical slice démontré
- Une preuve de qualité ou résilience
- Un dashboard ou coût analysé
- Une dette supprimée avant d'en créer une nouvelle

Annexe F - Sources officielles et veille

Références consultées pour figer la baseline au 23 août 2026. Avant votre implémentation, relisez les release notes et matrices de compatibilité.

- Angular: <https://angular.dev/> et blog Angular v22
- Spring Boot: <https://docs.spring.io/spring-boot/reference/> (4.1.1)
- Java: <https://www.oracle.com/java/technologies/downloads/> (Java 25 LTS)
- Flutter: <https://docs.flutter.dev/release/release-notes> (3.47)
- PostgreSQL: <https://www.postgresql.org/docs/release/18.6/>
- Apache Kafka: <https://kafka.apache.org/community/downloads/> (4.3.1)
- Keycloak: <https://www.keycloak.org/downloads> (26.7.2)
- Kubernetes: <https://kubernetes.io/releases/> (1.36.2)
- OAuth 2.0 Security BCP: RFC 9700
- Problem Details for HTTP APIs: RFC 9457
- W3C Trace Context: <https://www.w3.org/TR/trace-context/>
- OWASP ASVS, API Security Top 10 et MASVS
- CNCF OpenTelemetry: <https://opentelemetry.io/docs/>

Politique de veille

Chaque mois: vulnérabilités et patch releases. Chaque trimestre: dépendances, coûts, SLO et capacité. Tous les six mois: ADR structurants, roadmap de versions majeures et exercice de reprise. N'adoptez pas une nouveauté parce qu'elle est disponible; adoptez-la quand elle réduit un risque mesuré ou simplifie durablement le système.

Conclusion - Ce qui fait un bon architecte

Un bon architecte n'est pas la personne qui connaît le plus de frameworks. C'est celle qui rend les compromis explicites, réduit les risques tôt, préserve la capacité de changement et aide les équipes à livrer un système observable, sûr et économiquement viable.

Votre soutenance finale

- Démontrer onboarding d'un tenant, publication d'un produit, prix contractuel, commande, approbation et paiement sandbox.
- Couper Inventory ou Kafka et expliquer la dégradation, la reprise et les doublons.
- Prouver qu'un utilisateur du tenant A ne peut lire ni modifier les données du tenant B.
- Montrer une trace distribuée, un dashboard SLO, une alerte et son runbook.
- Restaurer une sauvegarde et vérifier RPO/RTO.
- Présenter cinq ADR avec alternatives, coûts, conséquences et signaux de révision.
- Expliquer le coût par tenant et par commande, puis proposer le prochain incrément.

Vous serez prêt à agir comme architecte lorsque vous pourrez défendre le système, le faire évoluer et l'opérer - pas seulement le faire compiler.